

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 4

Выполнил:
Епифанов Кирилл
К3343

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Введение

В рамках данной работы я спланировал процесс преобразования существующего backend-приложения в распределенную микросервисную систему с использованием подхода database-per-service, при котором каждый сервис обладает собственной независимой базой данных и отвечает только за свою предметную область.

Целью данной работы является проектирование полноценной микросервисной архитектуры, включающей:

- разделение монолитного приложения на независимые сервисы;
- проектирование взаимодействия между сервисами через REST API;
- выделение отдельных баз данных для каждого сервиса;
- разработку API Gateway как единой точки входа;
- описание внутренних и публичных эндпоинтов;
- формализацию запросов, ответов и возможных ошибок;
- определение этапов миграции от монолита к микросервисной архитектуре.

Разработанная архитектура ориентирована на дальнейшее масштабирование системы, повышение отказоустойчивости и упрощение сопровождения отдельных бизнес-доменов приложения.

Ход работы

Текущее состояние

- Домены вынесены в каталоги: auth-service, user-service, property-service (cities, facilities, properties), booking-service, messaging-service
- HTTP-маршруты сохранены в routing-controllers: /auth, /users, /cities, /properties, /facilities, /bookings, /properties/:propertyId/chats.
- Запуск по-прежнему монолитный (один express, один dataSource в app.ts). Межсервисные зависимости через импорты TypeScript, не через сеть.

Целевая микросервисная архитектура

- API Gateway (или BFF): единая точка для клиента, маршрутизация по сервисам, CORS, rate limit.
- Auth: регистрация, логин, выпуск/валидация JWT (секрет только здесь или через KMS).

- User: профиль, CRUD пользователей (внутренние read для других сервисов).
- Property: города, объекты, удобства, связь property–facility.
- Booking: брони; ссылки userId/propertyId как внешние идентификаторы без FK в чужие БД.
- Messaging: чаты и сообщения по propertyId.

Схема связей:

клиент=>Gateway=>сервисы. Booking вызывает Property GET /internal/properties/{id} (существование, владелец). Messaging вызывает User/Property для проверки участников. Опционально позже: события (Kafka/RabbitMQ) для уведомлений о бронировании.

БД на сервис

- db_auth: сессии/refresh при необходимости, таблицы только auth-домена
- db_user: users + хэш пароля
- db_property: cities, properties, facilities, property_facility.
- db_booking: bookings
- db_messaging: chats, messages.

Запрет: прямой SQL/join между БД.

Межсервисные REST (OpenAPI отдельными файлами или tags)

- user-service: GET /internal/users/{id}, GET /internal/users/by-email?email= (только из private network).
- property-service: GET /internal/properties/{id}, HEAD /internal/properties/{id}.
- auth-service: POST /internal/token/validate (body: token) => 200 {userId} / 401.
- Все internal: mTLS или shared service token, не публиковать наружу.

Публичные эндпоинты — как в текущем API; при разбиении — префиксы сервисов за Gateway без изменения контракта для клиента.

Запросы, ответы, ошибки (для спецификации)

- Успех: 200/201 + JSON DTO; 204 для delete без тела.
- 400: ошибки валидации (массив полей).
- 401: нет JWT.

- 403: JWT ок, нет прав (не владелец property/booking).
- 404: сущность не найдена.
- 409: конфликт (даты брони, дубликат email).
- 500: внутренняя ошибка, без утечки стека наружу.

Шаги миграции с монолита

- Выровнять импорты и один общий compose на время strangler: поднять 5 контейнеров Postgres + 5 сервисов + gateway.
- Для каждого сервиса: свой TypeORM DataSource, свои миграции, свой DATABASE_URL.
- Вынести проверку JWT в middleware каждого сервиса с вызовом auth /internal/token/validate или локальной проверкой подписи с публичным ключом.
- Заменить межмодульные импорты сущностей на HTTP-клиенты (retry, timeout).
- Разрезать данные: pg_dump по таблицам => import в целевые БД; downtime или dual-write поэтапно.
- Добавить /health и /ready на каждый сервис; централизованные логи (JSON) и трассировка (trace id через Gateway).

Плюсы и минусы

- Плюсы: независимое масштабирование, изоляция сбоев БД, границы команд.
- Минусы: распределенные транзакции, сложнее отладка, сетевые сбои, дублирование инфраструктуры.

Заключение

В результате выполнения работы была спроектирована микросервисная архитектура для существующего api с использованием подхода database-per-service. Были определены границы сервисов, выделены независимые предметные области и разработана схема взаимодействия между компонентами системы через REST API.